Dev Log — 26–28 July 2026

Session Goal

Finish the `System` / `GameSystemManager` redesign sketched in the 26 July design doc (`design-docs/2026-07-26_134754_system-and-game-system-managers.md`): give `Scene` real entity/game-system ownership wrappers, move `EntityManager` up to application lifetime, and land the actual spawn/growth game systems (`AppleSpawnSystem` , `SnakeSpawnSystem` , `SnakeGrowSystem`) that the previous session's plan called for. Three commits cover this arc — `7955fae` ("wip: game systems and systems implementation", 26 July), `5d97178` ("adding game systems", 27 July), and `af44c07` ("finish refactoring game systems", 28 July). As with recent sessions, all code was written by hand by the user per `CLAUDE.md` 's read-only-for-the-agent policy; Claude's role was prose design discussion (how to give game systems entity-creation capability without threading `IEntityManager` through every leaf class) and, at the end of this log's window, direct diagnosis — reading the actual current source and running a real build — rather than trusting the commit messages at face value. The 28 July commit message ("finish refactoring") is optimistic: the refactor's *shape* is in, but the build does not currently compile. See Known Issues.

Work Completed

`Scene` gained real `AddEntity` / `RemoveEntity` / `AddGameSystem` / `RemoveGameSystem` wrappers

`7955fae` first landed `Scene` as a new base class with private `entityManager` , `gameSystemManager` , and `eventBus` references, `AddEntity` / `AddGameSystem` wrappers that track what they create in `ownedEntities` / `ownedGameSystems` , and an `OnExit()` meant to tear all of it down — but shipped with a literally broken body (`for()` with no condition, no loop variable). `5d97178` filled in `OnExit()` to actually iterate both owned vectors and remove each entry. `af44c07` then added the still-missing `Scene::RemoveEntity` / `Scene::RemoveGameSystem` definitions (declared since `7955fae` but never defined), fixed `RemoveGameSystem` 's signature (it took `unique_ptr<IGameSystem>` by value, but `ownedGameSystems` only ever holds raw pointers — there was nothing to move from at any real call site) to take a raw pointer like `RemoveEntity` does, and routed `OnExit()` 's two loops through these wrappers instead of calling `entityManager` / `gameSystemManager` directly — one source of truth for "remove and untrack" instead of two diverging code paths.

`GameSystem` extended to own entity creation, mirroring `Scene`

`af44c07` 's core move: `GameSystem` 's constructor now takes `IEntityManager&` alongside `IEventBus&` , storing it privately and exposing public `AddEntity` / `RemoveEntity` wrappers plus a `GetEventBus()` accessor (the `eventBus` member moved from `protected` to `private` alongside it). This is the design worked through in conversation this session: `IEntityManager` should only ever be visible at the *wiring* points (`Scene` 's constructor, and — now — `GameSystem` 's), never as a field or constructor parameter on a

concrete game system. `Scene` also gained a `CreateGameSystem<T>(Args...)` template that constructs `T` with `this->eventBus` / `this->entityManager` supplied positionally, then routes the result through the existing `AddGameSystem`, so `GameplayScene::OnEnter` never has to name `IEntityManager` at all — it just calls `this->CreateGameSystem<SnakeGrowSystem>(snakeGrowParams)`.

Three real game systems landed: `AppleSpawnSystem`, `SnakeSpawnSystem`, `SnakeGrowSystem`

`5d97178` gave `AppleSpawnSystem` a real body for the first time (previously an empty stub with a `RespawnApple()` that did nothing) — it now constructs an `Apple` entity on `OnRegistration`, subscribes to `GameStartedEvent` / `AppleCollectedEvent`, and respawns by toggling `SetActive` around a `SetPosition` to a random point via the renamed `Core::Utils::GetRandomValue` (moved this commit from `core/random/get-random-value.hpp`). `SnakeSpawnSystem` went from an empty `.cpp` to a full implementation: `CreateHead` / `CreateBody` / `CreateSegment` / `TeardownBody` / `Reset`, replacing the logic that used to live on `Snake::CreateBody()` (deleted this commit). `SnakeGrowSystem` is new this session (`5d97178` added the header+stub, `af44c07` finished `Grow()`): on `AppleCollectedEvent`, it builds a new segment's transform from the current tail (or the head, if the body is still empty) and appends it via the new `SnakeBodyComponent::Append` (renamed from `AddBodyPart` in `af44c07`, for consistency with `std::deque`-style naming).

`EntityManager` promoted to an application-lifetime `System`

Per the design doc's stated goal ("`EntityManager` is scene-owned... comment already flagging it as needing to move up to `Application`"), `af44c07` registers a real `EntityManager` instance via `Application::RegisterSystem` in `Application::Initialize()`, alongside `GameSystemManager`. This retires the old pattern where `GameplayScene` constructed its own `EntityManager` in its constructor and manually called `entityManager->OnUpdate(deltaTime)` from `GameplayScene::Update()` (that call is now gone — ticking is `SystemManager`'s job, the same way `GameSystemManager` already worked).

Housekeeping

`af44c07` deleted a stray root-level file named `;` — a byte-identical duplicate of an early, broken `apple-spawn-system.cpp` (missing semicolon on `this->apple->SetActive(true)`, no closing brace), almost certainly created by an accidental shell redirect during a previous session rather than intentional. `entity.hpp` / `entity.cpp` picked up a real return value from `AddComponent<T>` (it used to return `void`; now returns `T*`), letting `Entity::OnRegistration()` capture `this->transform` directly from the call that creates it instead of a separate `GetComponent` lookup immediately after.

Lessons Learned

A DI container's `Register<Interface, Impl>()` is only as good as its construction strategy

Came up while discussing whether `IEntityManager` could be made "always available" to every `Scene` and `GameSystem` via the existing `DependencyInjector` rather than threaded through constructors by hand.

`IDependencyInjector::Register<TInterface, TImplementation>()` (`core/dependency-injection/i-dependency-injector.hpp:25-32`) always calls `std::make_shared<TImplementation>()` — no-arg construction only. `EntityManager`'s real constructor needs `(IRenderComponentManager*, ActionRouter&)`, so it can't go through that path as-is; a service-locator-style `Resolve<IEntityManager>()` call from inside `GameSystem`'s constructor was considered and rejected in favor of explicit constructor threading, since it would've been the only place in this codebase resolving a dependency implicitly rather than receiving it.

Ownership shape dictates the API shape: raw-pointer containers can't take a `unique_ptr` back

`Scene::RemoveGameSystem`'s original signature (`unique_ptr<IGameSystem>` by value) was a mismatch traced back to what `ownedGameSystems` actually stores — a `vector<IGameSystem*>` of raw, non-owning pointers (the real ownership lives in `GameSystemManager`). A "remove" method's parameter type should match what callers can actually produce from the container they're removing from, not assume the caller still holds a smart pointer once ownership has already moved elsewhere.

An engine-layer base class is allowed to see infrastructure a game-layer subclass shouldn't

The recurring design question this session — "how do I stop passing `IEntityManager` around in game code" — resolved to a layering distinction rather than a mechanism: `GameSystem` (engine namespace) is allowed to hold `IEntityManager&`, because it's the base class wrapping it into a narrower capability (`AddEntity` / `RemoveEntity`), exactly the same shape `Scene` already used for the same reason. Concrete game systems (`SnakeGrowSystem`, etc., `SnakeGame` namespace) were never meant to hold it themselves — only to forward it, unnamed, through to the base constructor.

Architectural Decisions

`Scene::CreateGameSystem<T>` centralizes shared-dependency wiring instead of a global registry

Discussed two options for making `eventBus` / `entityManager` "always available" to game systems without repeating `GetEventBus()` / `GetEntityManager()` at every construction call site: a DI-backed service locator, or a single templated factory on `Scene` that supplies both positionally before any system-specific args. Chose the factory — it keeps dependencies explicit in every constructor signature (`GameSystem(IEventBus&, IEntityManager&, ...)`), confines the one place that ever names `IEntityManager` in game-facing code to `Scene` itself, and doesn't require extending the DI container to support non-default-constructible registrations just for this.

Concrete game systems take shared deps as leading constructor params, not struct fields

`AppleSpawnSystemParams` / `SnakeSpawnSystemParams` / `SnakeGrowSystemParams` all dropped their `eventBus` / `entityManager` fields this session. Each constructor now takes `(IEventBus&, IEntityManager&, ThisSystem'sParams&)` — the first two forwarded straight into `GameSystem`'s initializer, the params struct carrying only what's actually specific to that system (`screenWidth` / `screenHeight` , `settings`). This keeps the params structs honest about what each system uniquely needs versus what every system gets for free from its base class.

`EntityManager` lives at application scope, ticked by `SystemManager` like everything else

Confirmed the design doc's premise: an `EntityManager` recreated per-scene can't hold anything that should survive a scene transition, and manually ticking it from `GameplayScene::Update()` was a workaround for it not being a registered `System` in the first place. Registering it via `Application::RegisterSystem` alongside `GameSystemManager` makes both consistent with how `InputSystem` already works, and removes a scene-specific special case.

Known Issues Carried Forward

- ☐ **UE-0052** — MAIN QUEST (build blocker, confirmed by direct build attempt): `Scene::Scene()` has no definition anywhere in `src/` . `Scene` (`engine/scenes/scene.hpp:19`) declares `explicit Scene();` and holds three **reference** members (`entityManager` , `gameSystemManager` , `eventBus`) that can only be initialized in a constructor's initializer list — there is no `Scene::Scene()` body in `scene.cpp` to do that. Every scene that inherits from `Scene` (currently just `GameplayScene` , whose own constructor never explicitly initializes the base) will fail to link the moment this constructor is actually needed.
- ☐ **UE-0053** — MAIN QUEST (build blocker, confirmed by direct build attempt): `Engine::Entities::EntityManager` no longer satisfies `IEntityManager` / `ISystem` . `af44c07` registers `make_unique<EntityManager>(...)` in `Application::Initialize()` (`application.cpp:87-91`), but `ISystem` picked up `IOnRegistration` back in `7955fae` (`core/systems/i-system.hpp`), and `EntityManager` never implements `OnRegistration()` — confirmed via an actual build: `cmake --build build` fails with "invalid new-expression of abstract class type `Engine::Entities::EntityManager` ", naming `Core::IOnRegistration::OnRegistration()` as the unimplemented pure virtual. This is the first thing that has to be fixed before anything else in this session's work can even compile.
- ☐ **UE-0054** — MAIN QUEST (build blocker, will surface right after UE-0053 is fixed): `AppleSpawnSystem` , `SnakeSpawnSystem` , and `SnakeGrowSystem` still reference `this->entityManager` directly (`apple-spawn-system.cpp:29` ; `snake-spawn-system.cpp:48,87,96,106` ; `snake-growth-system.cpp:36`), but `af44c07` moved `GameSystem::entityManager` (`engine/systems/game-systems/game-system.hpp`) to `private` and removed each system's own `entityManager` field —

none of these three call sites were migrated to the new public `this->AddEntity(...)` / `this->RemoveEntity(...)` wrappers `GameSystem` now provides. This is exactly the mechanical follow-through discussed in this session's design conversation; it just hasn't been applied to the actual call sites yet.

- [ ] **UE-0055** — MAIN QUEST: `GameplayScene::snake` is a `unique_ptr<Snake>` that is never constructed anywhere, but `GameplayScene::Update()` (`gameplay-scene.cpp`) unconditionally calls `this->snake->Update(deltaTime)` every frame — a guaranteed null-pointer dereference once the build actually links. This predates this session (the previous log already noted `Snake` 's movement logic was mid-migration to `SnakeSegment` / `SnakeHead`), but with `SnakeSpawnSystem` and `SnakeGrowSystem` now owning spawn/grow responsibility entirely, `Snake` itself (`game-entities/snake.hpp` / `.cpp`) looks like dead code rather than something to finish migrating: its `Initialize()` still assigns `this->length` even though that member was deleted from `Snake` in `5d97178` ; `GetCenter()` has no return statement; `CheckIfShouldGrow()` and `Teleport()` are fully commented out. Recommend confirming next session whether `Snake` and the `GameplayScene::snake` member should simply be deleted, rather than treating this as more migration work.
- [ ] **UE-0009** — Flagging for confirmation before closing: `AppleSpawnSystem` is now a real, wired implementation (event-subscribed respawn, random position via `Core::Utils::GetRandomValue`), which reads as this ticket ("Implement `AppleSpawner` ") being resolved. Holding off on closing until the build issues above (UE-0052/53/54) are fixed and the system can actually be confirmed running, rather than closing on diff evidence alone.
- [ ] **UE-0004** — BUG: Window-side event bus stub. Unchanged.
- [ ] **UE-0014** — REAPER: `GameplayState` still owns stale snake/apple references and polls input. Unchanged.
- [ ] **UE-0015** — REAPER: `UserInterfaceManager` update/debug methods commented out. Unchanged.
- [ ] **UE-0016** — REAPER: `SnakeGameSettings::GetBoxSize()` commented out. Unchanged.
- [ ] **UE-0019** — SIDE QUEST: `BoundaryWrapSystem` . Unchanged — `Snake::Teleport()` still has the wrapping logic fully commented out with the same explanatory note as last session, and per UE-0055 above, may end up moot if `Snake` itself is retired rather than finished.
- [ ] **UE-0020**/**UE-0023** — REAPER: Remove stale `stateMachine` `shared_ptr` from `Application` members. Not investigated this session.
- [ ] **UE-0021** — 1UP: `std::is_abstract_v` discriminator breaks for `ColliderComponent2D` . Not investigated this session.
- [ ] **UE-0024** — REAPER: UI component system commented out. Not investigated this session.
- [ ] **UE-0026** — REAPER: Remove `Apple` verification hooks before merge. Not investigated this session.

- [ ] **UE-0028** — MAIN QUEST: `GameplayStateMachine` use-after-move. Unchanged, still entangled with UE-0035.
- [ ] **UE-0029** — RAID BOSS: Unify entity/component ownership to remove the leak and double free. Unchanged in scope this session.
- [ ] **UE-0030** — 1UP: `TransformComponent2D::GetTransform()` returns a shared function-local `static`. Not touched this session.
- [ ] **UE-0031** — REAPER: Dependency injector correctness bugs. Not touched this session (see Lessons Learned for a related but distinct observation about `Register<>`'s construction strategy — not itself a bug, just a limitation worth remembering).
- [ ] **UE-0032** — REAPER: `ComponentDispatcher` `const_cast`ing. Not touched this session.
- [ ] **UE-0033** — REAPER: Miscellaneous cleanups from code review. Not touched this session.
- [ ] **UE-0036** — Event's `action` field never populated. Still unconfirmed two sessions running; recommend resolving the flag rather than carrying a third time.
- [ ] **UE-0037** — Keymap array literal order didn't match `Action` enum order. Same recommendation as UE-0036.
- [ ] **UE-0050** — SIDE QUEST: Wire `showClientDebugLogs` into the logger properly. Not touched this session.
- [ ] **UE-0051** — RAID BOSS: Debug System needs a full refactor. Not touched this session.

Next Session

- [ ] **UE-0052** — Fix `Scene::Scene()` (see Known Issues — this blocks the build entirely).
- [ ] **UE-0053** — Implement `EntityManager::OnRegistration()` (see Known Issues — blocks the build immediately after UE-0052).
- [ ] **UE-0054** — Migrate `AppleSpawnSystem` / `SnakeSpawnSystem` / `SnakeGrowSystem` off `this->entityManager` onto `this->AddEntity` / `this->RemoveEntity` (see Known Issues — blocks the build immediately after UE-0053).
- [ ] **UE-0055** — Decide `Snake`'s fate: finish the migration or delete the class and `GameplayScene::snake` outright (see Known Issues).
- [ ] **UE-0048** — MAIN QUEST: Carried forward, now folded into UE-0055's scope — the per-segment movement migration and `Snake`'s own half-commented-out methods are the same underlying question of what, if anything, `Snake` should still own.
- [ ] **UE-0049** — MAIN QUEST: Implement the latched `pendingDirection` command queue. Still blocked on UE-0055/UE-0048 landing first, since the tick-boundary block this depends on lives in code

that's either mid-rewrite or slated for deletion.

- [ ] **UE-0038** — MAIN QUEST: Migrate `Confirm` / `Decline` off the dead `InputSystem::IsActionPressed` API. Unchanged this session.
- [ ] **UE-0035** — MAIN QUEST: `GameContext::input` is never assigned; reconcile the `MainMenuState` workaround. Unchanged, still blocked on UE-0038 (plus UE-0014, UE-0028).